ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Cross-Exchange Spread Scanner

Spot vs Futures — Binance × Bybit

Проект: bali
Модуль: spread-scanner
Версия: 1.0
Дата: 15 марта 2026

1. Цель и контекст

1.1 Бизнес-задача

Построить модуль cross-exchange spread scanner, который в реальном времени вычисляет разницу (спред) между спотовой ценой одной биржи и фьючерсной ценой другой биржи. Модуль фильтрует только ситуации, когда спот дешевле фьючерсов (контанго / положительный базис), генерирует сигналы и публикует их в Redis + файловый лог.

1.2 Место в проекте bali

Модуль является надстройкой над существующим слоем market-data, который уже собирает best bid/ask с четырёх WS-стримов (binance_spot, binance_futures, bybit_spot, bybit_futures) и пишет их в Redis в формате HSET md:{exchange}:{market}:{SYMBOL}.

1.3 Два направления сканирования

Направление	Спот (bid)	Фьючерс (ask)	Формула
A → B	Binance Spot best_bid	Bybit Futures best_ask	(bybit_fut_ask − binance_spot_bid) / binance_spot_bid × 100%
B → A	Bybit Spot best_bid	Binance Futures best_ask	(binance_fut_ask − bybit_spot_bid) / bybit_spot_bid × 100%

Фильтр: сигнал генерируется только если spread_pct > 0 (спот дешевле фьючерса). Отрицательные спреды (бэквордация) игнорируются.

2. Архитектура

2.1 Принцип работы (максимально простая логика)

Один Python-процесс spread_scanner.py с единственным asyncio event-loop:

1. Тикер-цикл: каждые SCAN_INTERVAL_MS (по умолчанию 200 мс) пробегает по списку символов.
2. Для каждого символа: два HGETALL из Redis (spot + futures с разных бирж) — используем pipeline для батч-чтения.
3. Вычисление spread_pct по формуле из п. 1.3.
4. Если spread_pct > MIN_SPREAD_PCT → запись сигнала в Redis + лог.
5. Обновление Prometheus-метрик.

Никаких отдельных потоков, очередей, Kafka, gRPC. Один процесс, один цикл, чтение из Redis, запись в Redis.

2.2 Структура файлов

```
bali/
├── spread-scanner/
│   ├── spread_scanner.py      # Главный модуль (вся логика в одном файле)
│   ├── config.py              # Конфигурация из .env
│   ├── .env.example
│   ├── requirements.txt
│   └── tests/
│       ├── test_spread_calc.py
│       └── test_signal_format.py
├── run.py                     # Добавить spread_scanner в процессы
└── logs/
    └── spread_scanner.log
```

2.3 Интеграция с run.py

Добавить spread_scanner в список процессов run.py. Должен запускаться командой:

```
python3 run.py                          # все процессы вкл. сканер
```

3. Чтение данных из Redis

3.1 Формат ключей (уже существует)

Ключ Redis	Поля
md:binance:spot:{SYMBOL}	best_bid, best_ask, best_bid_qty, best_ask_qty, ts
md:binance:futures:{SYMBOL}	best_bid, best_ask, best_bid_qty, best_ask_qty, ts
md:bybit:spot:{SYMBOL}	best_bid, best_ask, best_bid_qty, best_ask_qty, ts
md:bybit:futures:{SYMBOL}	best_bid, best_ask, best_bid_qty, best_ask_qty, ts

3.2 Батч-чтение через pipeline

Для списка из N символов за один цикл нужно 4×N HGETALL. Использовать redis.asyncio pipeline для отправки всех команд одним batch:

```
pipe = redis.pipeline(transaction=False)
for symbol in symbols:
    pipe.hgetall(f'md:binance:spot:{symbol}')
    pipe.hgetall(f'md:bybit:futures:{symbol}')
    pipe.hgetall(f'md:bybit:spot:{symbol}')
    pipe.hgetall(f'md:binance:futures:{symbol}')
results = await pipe.execute()  # один RTT
```

3.3 Валидация свежести

Перед расчётом проверять: abs(time.time() - float(ts)) < STALE_THRESHOLD_SEC (по умолчанию 5 сек). Если данные устарели — пропускаем символ, инкрементируем метрику stale_skips_total.

4. Вычисление спреда

4.1 Формула

```
spread_pct = (futures_ask - spot_bid) / spot_bid * 100
```

Где futures_ask — best_ask фьючерса, spot_bid — best_bid спота. Всегда берём ask фьючерса (цена покупки) и bid спота (цена продажи), потому что это реальные цены исполнения для арбитража.

4.2 Условие сигнала

```
if spread_pct > MIN_SPREAD_PCT:  # дефолт 0.05%
    emit_signal(...)
```

Отрицательные спреды (бэквордация) — полностью игнорируются, никуда не пишутся.

5. Формат сигнала

5.1 JSON-структура сигнала

```
{
  "ts": 1710504000.123,
  "symbol": "BTCUSDT",
  "direction": "binance_spot_to_bybit_futures",
  "spot_exchange": "binance",
  "futures_exchange": "bybit",
  "spot_bid": "67432.10",
  "futures_ask": "67498.50",
  "spread_abs": "66.40",
  "spread_pct": "0.0985",
  "spot_bid_qty": "1.234",
  "futures_ask_qty": "0.500"
}
```

5.2 Запись в Redis

Два механизма записи:

a) HSET — последнее актуальное состояние (перезаписывается):

```
HSET sig:spread:{direction}:{SYMBOL} <все поля сигнала>

Пример ключа:
sig:spread:binance_spot_to_bybit_futures:BTCUSDT
sig:spread:bybit_spot_to_binance_futures:ETHUSDT
```

b) Redis Pub/Sub — для подписчиков в реальном времени:

```
PUBLISH ch:spread_signals <JSON сигнала>
```

TTL на HSET ключах: 60 секунд (автоочистка если сканер упал).

5.3 Запись в лог-файл

Файл: logs/spread_signals.log (отдельно от основного лога сканера).

Формат: по одной JSON-строке на сигнал (JSONL), с ротацией 50 МБ, 5 файлов.

```
{"ts":1710504000.123,"symbol":"BTCUSDT","direction":"binance_spot_to_bybit_futures"
,...}
{"ts":1710504000.456,"symbol":"ETHUSDT","direction":"bybit_spot_to_binance_futures"
,...}
```

6. Конфигурация

6.1 Переменные окружения (.env)

Переменная	По умолчанию	Описание

REDIS_URL	redis://localhost:6379/0	Подключение к Redis
SCAN_INTERVAL_MS	200	Интервал сканирования (мс)
MIN_SPREAD_PCT	0.05	Минимальный спред для сигнала (%)
STALE_THRESHOLD_SEC	5	Макс. возраст данных в Redis (сек)
SIGNAL_TTL_SEC	60	TTL ключей сигналов в Redis
METRICS_PORT	9091	Порт Prometheus-метрик
LOG_LEVEL	INFO	Уровень логирования

6.2 Список символов

Использовать существующий файл dictionaries/subscribe/common_symbols.json (пересечение пар доступных на обеих биржах для spot и futures). Загружается один раз при старте. Горячая перезагрузка: по SIGHUP.

7. Метрики (Prometheus)

Экспозиция: prometheus_client, HTTP на порту METRICS_PORT.

Метрика	Тип	Labels	Описание
spread_scanner_cycle_duration_seconds	Histogram	—	Время полного цикла сканирования
spread_scanner_signals_total	Counter	direction, symbol	Количество сгенерированных сигналов
spread_scanner_spread_pct	Gauge	direction, symbol	Текущий спред в % (обновляется каждый цикл)
spread_scanner_stale_skips_total	Counter	exchange, market	Пропуски из-за устаревших данных
spread_scanner_redis_errors_total	Counter	—	Ошибки подключения к Redis
spread_scanner_symbols_active	Gauge	—	Кол-во символов в текущем цикле
spread_scanner_pipeline_rtt_seconds	Histogram	—	Время pipeline HGETALL

Важно: для labels direction/symbol использовать ограниченный набор (не создавать высококардинальные метрики). Для spread_pct gauge — хранить только top-N символов или только те, где был сигнал.

8. Оптимизация и производительность

8.1 Требования к скорости

Параметр	Целевое значение
Полный цикл (100 символов)	< 50 мс
Redis pipeline RTT	< 5 мс (localhost)
CPU при 200 мс интервале	< 5% одного ядра
RAM	< 50 МБ

8.2 Ключевые оптимизации

- Один Redis pipeline на весь цикл (не по одному HGETALL).
- Переиспользование aioredis connection pool (не создавать новое подключение).
- float() конвертация сразу при парсинге, хранить в dict, не создавать объекты.
- Минимум аллокаций: переиспользовать буферы для JSON-сериализации сигналов.

- orjson вместо json для сериализации (в 3-5 раз быстрее).
- Асинхронная запись в лог (не блокировать цикл на I/O).
- Никаких deepcopy, dataclass, Pydantic в hot-path.

8.3 Чего НЕ делать

- Не использовать SUBSCRIBE для чтения данных (polling через pipeline быстрее и предсказуемее).
- Не создавать отдельные потоки/процессы для каждого направления.
- Не хранить историю спредов в памяти (только последнее значение).
- Не использовать ORM, dataclass, или тяжёлые абстракции в основном цикле.

9. Обработка ошибок

Ситуация	Поведение
Redis недоступен	Retry с exp. backoff (1s, 2s, 4s... до 30s). Метрика redis_errors_total++. Лог ERROR.
Данные по символу отсутствуют	Пропустить символ, stale_skips_total++. Лог DEBUG.
Данные устарели (ts > STALE_THRESHOLD)	Пропустить символ, stale_skips_total++. Лог WARNING.
Невалидные данные (не парсится float)	Пропустить символ, лог WARNING. Не крашить процесс.
Ошибка записи сигнала в Redis	Лог ERROR, продолжить цикл. Не терять данные в логе.
SIGHUP (перезагрузка символов)	Перечитать файл символов, лог INFO.
SIGTERM/SIGINT	Graceful shutdown: закрыть Redis pool, финализировать логи.

10. Тесты

10.1 Unit-тесты (pytest)

Тест	Что проверяет
test_spread_positive	spread_pct корректен при spot < futures
test_spread_negative_ignored	Отрицательный спред не генерирует сигнал
test_spread_below_threshold	Спред ниже MIN_SPREAD_PCT не генерирует сигнал
test_stale_data_skip	Устаревшие данные пропускаются
test_signal_json_format	JSON сигнала содержит все обязательные поля
test_redis_key_format	Ключи сигналов формируются правильно
test_pipeline_batch	Pipeline содержит 4×N команд для N символов

10.2 Acceptance test

Bash-скрипт tests/acceptance_check.sh:

1. Запустить spread_scanner на 5 секунд.
2. Проверить наличие ключей sig:spread:* в Redis.
3. Проверить наличие строк в logs/spread_signals.log.
4. Проверить доступность /metrics на порту METRICS_PORT.

11. Критерии приёмки

#	Критерий	Как проверить
1	Сканер стартует через run.py без ошибок	python3 run.py --only spread_scanner
2	Redis содержит сигналы sig:spread:*	redis-cli KEYS 'sig:spread:*'
3	Лог spread_signals.log содержит JSONL-строки	head logs/spread_signals.log | python3 -m json.tool
4	Prometheus-метрики доступны	curl localhost:9091/metrics
5	Полный цикл < 50 мс при 100 символах	Метрика spread_scanner_cycle_duration_seconds
6	Устаревшие данные не порождают сигналы	Unit-тест test_stale_data_skip
7	Отрицательные спреды игнорируются	Unit-тест test_spread_negative_ignored
8	Graceful shutdown по SIGTERM	kill -TERM <pid>, проверить что лог закрыт
9	pytest -q проходит без ошибок	cd spread-scanner && pytest tests/ -q

10	Код проходит ruff check без замечаний	ruff check spread-scanner/

12. Зависимости (requirements.txt)

```
redis[hiredis]>=5.0
orjson>=3.9
prometheus-client>=0.20
python-dotenv>=1.0
```

Hiredis — C-расширение для парсинга ответов Redis, ускоряет pipeline в 2-3 раза.

13. Вне скоупа (не делать)

- Автоматическое исполнение ордеров (только сигналы).
- Docker / docker-compose (как и в основном проекте).
- Веб-интерфейс / дашборд (Grafana подключается к Prometheus самостоятельно).
- Хранение исторических спредов в БД (только текущее состояние + лог).
- Работа с другими биржами кроме Binance и Bybit.
- Coin-margined фьючерсы (только USDT-margined / linear).